An Effective Controller Placement Algorithm Based on Clustering in SDN

Rong Yujie, Wu Muqing, Chen Yiming
Beijing University of Posts and Telecommunications
School of Information and communication Engineering
Beijing Laboratory of Advanced Information Networks
Beijing Key Laboratory of Network System Architecture and Convergence Acknowledgment
Beijing, China
e-mail: {ryj, wumuqing}@bupt.edu.cn; j2044185473@163.com

2020 IEEE 6th International Conference on Computer and Communications (ICCC) | 978-1-7281-8635-1/20/$31.00 ©2020 IEEE | DOI: 10.1109/ICCC51575.2020.9345045

***Abstract*—Different from the traditional TCP/IP network, Software Defined Network (SDN) uses a layered architecture to decouple the control plane and data plane. The controller is the core device of the control plane, and the control plane is mainly responsible for the decision-making control of the routing strategy. Because the processing power of a single controller is limited, the multi-controller network architecture has become an inevitable choice in the WAN. The layout of the controllers in SDN has an impact on the performance of the entire network. The controller placement problem (CPP) is an NP-hard problem. The challenges of CPP solution are as follows. 1) the number of controllers; 2) the position of the controllers and the position of the switches under their management in the network topology. The clustering by fast search and find of density peaks (CFSFDP) algorithm is a density-based clustering algorithm. It has few initial parameters and does not require iterative solution. However, the important parameter d_c of the CFSFDP algorithm is usually selected based on experience, and the number of cluster centers cannot be determined automatically. This paper proposes a new controller placement algorithm by improving the CFSFDP algorithm. We use the idea of information entropy to automatically determine the d_c value, and automatically determine the number and location of the controllers based on the jumping point in the decision graph. It can be seen from the experimental results that compared with random placement, k-means and CFSFDP algorithm, our proposed algorithm can effectively reduce network propagation delay and improve network performance.**

Keywords-SDN; clustering algorithm; controller placement problem;

I. INTRODUCTION

In the traditional TCP/IP network architecture, the network equipment is usually developed and maintained by equipment manufacturers, and the equipment manufacturers are relatively closed, which make the types of network software diverse and poor compatibility with each other. In order to improve network flexibility and programmability, Software Defined Network (SDN) is proposed. SDN uses a layered architecture to decouple the control plane and data plane. SDN control plane is mainly responsible for the decision-making control of the routing strategy. The controller is the core device of the control plane and has global information about the network. The data forwarding function is completed by the data plane in SDN. The data plane is mainly composed of SDN switches, and SDN switches rely on forwarding tables to forward data. SDN realizes network programmable function through open user programming interface. Network administrators can configure the network according to their specific business by programming, so that they can manage the network flexibly.

At the beginning of the design, SDN has only one controller. Usually, a single controller can meet performance requirements in small networks. However, the processing power of a single controller is limited. So the global single-controller architecture can no longer meet the performance requirements in the WAN, and the multi-controller network architecture has become an inevitable choice [1]. At present, there are multiple distributed controller architectures, such as HyperFlow [2], Onix [3], etc. Each controller is responsible for managing switches in its domain, and the controllers coordinate with each other to make sure the entire network state is consistent. The controller placement problem (CPP) is an NP-hard problem. The key points of CPP solution are as follows. 1) the number of controllers; 2) the position of the controllers and the position of the switches under their management in the network topology.

There are many kinds of controller placement algorithms, such as greedy search algorithm, clustering algorithm, heuristic search algorithm, game algorithm, etc. For example, the authors in [4] apply the greedy search algorithm to the CPP solution. The paper [5] uses a bargaining game algorithm to solve the CPP, and solves the CPP through the continuous game of time delay, transmission load between controllers, and controller loads. The authors in [6,7,8] apply the simulated annealing algorithm in the heuristic random search algorithm to the CPP solution. The paper [9] applies the clustering algorithm to the CPP, and the authors use the clustering algorithm to divide the entire network into multiple subnets. The paper [10] proposes the clustering by fast search and find of density peaks (CFSFDP) algorithm, which has fewer initial parameters. Compared with other density clustering algorithms, the CFSFDP algorithm does not require iterative solution, and this is why it has faster execution speed. However, the important parameter d_c of the CFSFDP algorithm is usually selected based on experience. The parameter cannot be automatically determined, and the number of cluster centers cannot be determined automatically. Aiming at the CPP, this paper improves the CFSFDP

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:01:24 UTC from IEEE Xplore. Restrictions apply.

algorithm. For the problem of d_c relying on experience selection, our proposed algorithm uses the idea of information entropy to automatically determine the d_c value. For the problem that the CFSFDP algorithm cannot automatically determine the number of cluster centers, our proposed algorithm automatically calculates the number and location of controllers based on the jump characteristics from non-cluster centers to cluster centers [10].

The rest of paper is organized as follows. In Section II, the current research situation of clustering algorithms is introduced. In Section III, we improved the CFSFDP algorithm in the controller placement scenario. In Section IV, we evaluate the performance of the proposed algorithm on propagation delay. In Section V, we summarize the overall work.

II. Related Work

The principle of the clustering algorithm is to divide the data into multiple categories according to the inherent similarity, so that the similarity within the cluster is large, and the similarity between the clusters is small. Clustering algorithms have broad application prospects. There are many types of clustering algorithms, such as grid-based clustering algorithms, density-based clustering algorithms, model-based clustering algorithms, etc. The k-means clustering algorithm is a kind of density clustering algorithm. The k-means clustering algorithm initially randomly selects the object as the cluster center, and assigns each object to the closest cluster center. Every time a sample is allocated, the cluster centers need to be recalculated. This process continues to iterate until a certain termination condition is met. The CFSFDP algorithm is a kind of density algorithm and it does not require iteration. Each data point in this algorithm has two attributes: local density ρ and distance δ. We choose points with larger ρ and δ values as the cluster centers to ensure that the density of cluster centers is large and the distance between cluster centers is also large. The non-cluster center points are classified into the cluster where the nearest cluster center is located.

The CFSFDP algorithm currently has two problems. First of all, d_c is an important parameter for calculating local density. This parameter is generally determined based on experience, which is subjective and cannot guarantee the validity of the parameter. Secondly, the CFSFDP algorithm will draw a decision graph with ρ as the abscissa and δ as the ordinate, and manually select the points with larger ρ and δ values as the cluster centers according to the decision graph. The selection of cluster centers is also subjective.

In the scenario of the CPP, to solve the problem that the selection process of d_c is subjective, our proposed algorithm uses the idea of information entropy. We uses the local density to construct the information entropy function, then uses the firefly algorithm (FA) to find the value of d_c, which can maximize the information entropy. So we can increase the proportion of δ in decision-making. To solve the problem that the cluster centers cannot be determined automatically, we determine the jumping point by the distance from the point to the straight line, thereby determining the cluster centers.

III. Algorithm Formulation

The proposed algorithm is based on the CFSFDP algorithm to solve the CPP. We choose nodes with larger ρ and δ values as cluster centers to reduce the propagation delay. This means that the density of cluster centers is high, and there are many nodes around the cluster centers. Each cluster center has a relatively large distance from nodes with higher density. The location of the cluster center is the location where the controller is placed, and the number of cluster centers is also the number of controllers. The switches correspond to the closest controller. Each controller manages all switches in its domain. We use the undirected weighted graph $G(V,E)$ to represent the SDN network topology, where the network node is represented by V and the network link is represented by E. n is the number of nodes and $n = |V|$.

A. The determination of d_c

Algorithm1: The process of determining d_c

Input: *G=(V,E)*, distance matrix *D*.

1. Construct the FA lightness function according to formula (2) and (4).
2. Initialize the parameters of the firefly algorithm.
3. Initialize the position of each firefly, and calculate the lightness of each firefly according to the function.
4. while(*t*< *tMax*)
5. for *i* =1:*N*
6. for *j* =1:*N*
7. if *Lightn(j)>Lightn(i)*
8. According to formula (5)(6)(7), move the firefly *i* to the direction of *j*.
9. Update the lightness of firefly *i*.

output: cutoff distance d_c

We can use the cut-off kernel or the Gaussian kernel to compute the local density value.

- Cut-off kernel

$$\rho_i = \sum_{j \in V/\{i\}} \chi(d_{ij} - d_c) \tag{1}$$

where $\chi(x) = \begin{cases} 1, x < 0; \\ 0, x \geq 0. \end{cases}$, the distance between switch *i* and *j* is d_{ij}.

- Gaussian kernel

$$\rho_i = \sum_{j \in V/\{i\}} e^{-(\frac{d_{ij}}{d_c})^2} \tag{2}$$

It can be seen that the cut-off kernel is a discrete value, and the Gaussian kernel is a continuous value. Using the Gaussian kernel to calculate ρ value can reduce the probability that different points have the same ρ value. The proposed algorithm uses the Gaussian kernel density. In this paper, d_c is an important parameter for calculating local density, and this parameter is generally determined based on experience.

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:01:24 UTC from IEEE Xplore. Restrictions apply.

Usually we sort the distance between nodes in descending order, and the value in the range of 2% to 5% in the ascending sequence is taken as d_c [11]. According to the theory, the choice of cluster centers is more dependent on the value of δ [11], so we use ρ to construct the information entropy function, and take the d_c value that maximizes the information entropy to increase the proportion of δ in decision. Therefore, we can change the distribution of the decision graph to find better cluster centers. Because the formula (4) is a non-linear function, so our proposed algorithm uses FA to find the d_c value that maximizes the information entropy, and achieve the purpose of automatically determining d_c.

$$H(d_c) = -\sum_{i=1}^{n} \frac{\rho_i}{Z} ln\left(\frac{\rho_i}{Z}\right) \tag{3}$$

where $Z = \sum_{i=1}^{n} \rho_i$, representing the local density sum of all nodes.

$$d_c = arg\ \max_{d_c} \left\{H(d_c) = -\sum_{i=1}^{n} \frac{\rho_i}{Z} ln\left(\frac{\rho_i}{Z}\right)\right\} \tag{4}$$

Fireflies do not consider gender, and one firefly can attract all fireflies. The brighter the firefly, the more attractive it is. The low-lightness fireflies will be attracted by the high-lightness fireflies and move in their direction. At the same time, the light transmission medium between the fireflies absorbs light, so as the spatial distance increases, the attraction between the fireflies will decrease.

The firefly algorithm is as follows.

1) Initialize the position of each firefly and set the algorithm parameters. The position of each firefly in the D-dimensional space is $Y = (y_1, y_2, \cdots, y_D)$, set the number of firefly population N, the light absorption coefficient γ of the medium, the initial step size α, the maximum number of iterations $tMax$, the randomness reduction factor θ and the initial attractiveness β_0. The attractiveness formula is shown in formula (5).

$$\beta = \beta_0 e^{-\gamma r^2} \tag{5}$$

where β_0 is the attractiveness when the distance between two fireflies is 0, and r is the distance between two fireflies.

The step length is shown in formula (6).

$$step = \alpha\theta^t \tag{6}$$

where t is the number of iterations.

2) Calculate the lightness of each firefly based on the location of the firefly. The higher the lightness, the more attractive the firefly.

3) Each firefly flies in the direction of the firefly whose lightness is higher than itself, and its moving distance is shown in formula (7).

$$Y_i = Y_i + \beta\left(Y_j - Y_i\right) + step \tag{7}$$

4) Calculate the brightness of the new position the firefly reaches after flying to other fireflies whose brightness is higher than its own.

5) If the number of iterations reaches the maximum, the searched optimal firefly position will be output as the solution, otherwise it will skip to step 3.

B. The Determination of the Number and Location of Controllers

Algorithm2: The process of automatically determining the number and location of controllers

Input: *G=(V,E)*, calculated d_c, distance matrix *D*.

1. Calculate ρ based on d_c.
2. Sort the local density of the nodes in descending order to get the descending sequence (8).
3. Calculate δ_i of each node based on formula (9).
4. $\gamma_i = \rho_i * \delta_i$.
5. Normalize the γ values, and arrange the γ values in descending order: $\gamma_{s1} \geq \gamma_{s2} \geq \cdots \geq \gamma_{sn}$, to obtain the sequence $\{s_1, s_2, \cdots, s_n\}$.
6. Connect points s_1, and s_n to construct a straight line AB in the decision graph.
7. for each point in the decision graph
8. Calculate the distance VD_i from point s_i to AB.
9. Find the point with the largest VD value, this point is the jumping point.
10. The position of the point where the γ value is greater than the γ value of the jumping point is regarded as the placement position of the controller. The number of points whose γ value is greater than the γ value of the jumping point is regarded as the number of controllers. The switches correspond to the closest controller.

output: the number of controllers K, the location of the controllers *P* and the set of switches corresponding to the controllers.

After calculating ρ values of all nodes using the determined d_c value, we sort the ρ values of the nodes in descending order to obtain a descending sequence (8):

$$\rho_{q1} \geq \rho_{q2} \geq \cdots \geq \rho_{qn} \tag{8}$$

The distance δ_i is shown in formula (9).

$$\delta_{qi} = \begin{cases} \min\limits_{j<i} d_{qiqj}, i \geq 2; \\ \max\limits_{j\geq 2} d_{qiqj}, i = 1. \end{cases} \tag{9}$$

For the node x_i with the highest ρ value, its distance δ_i is the maximum distance from other nodes. In addition, the distance δ_i of other nodes x_i is the shortest distance from this node to other nodes with larger local density values. Aiming at the problem that the cluster center cannot be automatically selected, our proposed algorithm multiplies the ρ and δ

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:01:24 UTC from IEEE Xplore. Restrictions apply.

values to obtain the γ value, that is, $\gamma_i = \rho_i * \delta_i$. According to the theory: from non-cluster center to cluster center in the decision graph, there is an obvious jump in the value of γ_i. We normalize the γ value, then sort the γ values in descending order: $\gamma_{s1} \geq \gamma_{s2} \geq \cdots \geq \gamma_{sn}$, to obtain the sequence $\{s_1, s_2, \cdots, s_n\}$. Then we draw a decision graph, as shown in Fig. 1. It can be seen that there are jumping points from the non-cluster center to the cluster center. As shown in Fig. 2, we connect the points with the largest and smallest γ values in the decision graph as a straight line AB to form a triangle-like figure. It can be seen that the jumping point is the farthest from the straight line AB. So we can find the jumping point by finding the point with the largest distance from the line. The position of the point where the γ value is greater than the γ value of the jumping point is regarded as the placement position of the controller. The number of points whose γ value is greater than the γ value of the jumping point is regarded as the number of controllers. The switches correspond to the closest controller.

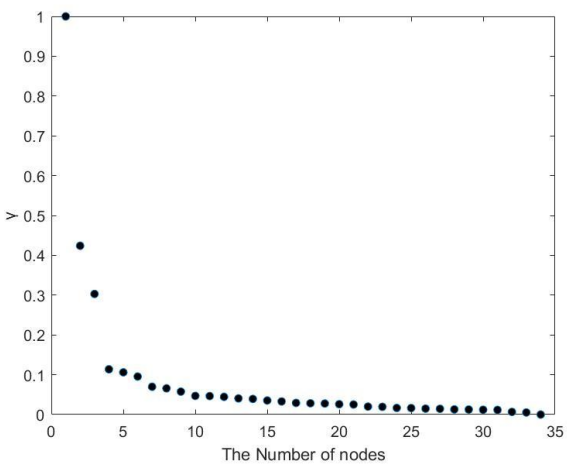

Figure 1. The descending order of r values

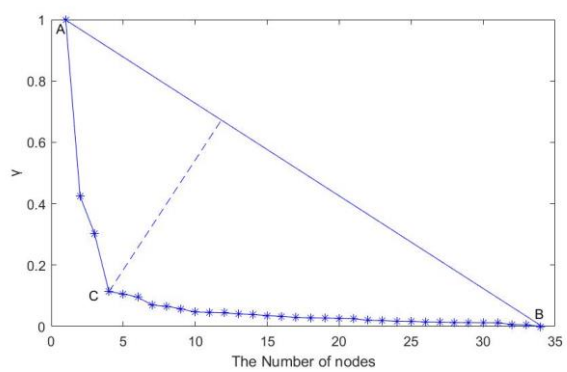

Figure 2. Connect the points with the largest and smallest γ values as a straight line AB

IV. Simulations and Results

We use the MATLAB simulation software to compare our proposed algorithm with random placement, k-means and CFSFDP algorithm. We use Internet2 OS3E real network data to conduct simulation experiments to verify the effectiveness of the algorithm. We use distance to characterize the delay and mainly simulate and verify the performance of the algorithm from the perspective of propagation delay.

A. Algorithm Evaluation

1) Average Delay in the Intra-controller: In the entire network, the average propagation delay between all switches managed by the same controller is shown in formula (10).

$$T_K^{avg} = \frac{1}{\sum_{k=1}^{K} \frac{1}{|V_k| \cdot (|V_k|-1)/2}} \sum_{k=1}^{K} \sum_{j>i}^{|V_k|} \sum_{i=1}^{|V_k|-1} d(i,j) \quad (10)$$

Where K represents the number of SDN controllers in the entire network, $|V_k|$ is the number of switches in the kth controller domain. $d(i,j)$ is the propagation delay between switch i and j, which is managed by the same controller.

2) Maximum Delay in the Intra-controller: In the entire network, the maximum propagation delay between all switches managed by the same controller is shown in formula (11).

$$T_K^{max} = \max_{k \epsilon \{1,2,\cdots,K\}, i \in V_k, j \in V_k / i} d(i,j) \quad (11)$$

3) Average Delay in the Inter-controller: In the entire network, the average propagation delay between all switches managed by the different controllers is shown in formula (12).

$$T^{avg} = \frac{1}{2} \sum_{k=1}^{K} \frac{1}{|V_k||V/V_k|} \sum_{j \in V/V_k} \sum_{i \in V_k} d(i,j) \quad (12)$$

Switch i and j belong to different clusters, which also means that they belong to the different controllers.

4) Maximum Delay in the Inter-controller: In the entire network, the maximum propagation delay between all switches managed by the different controllers is shown in formula (13).

$$T^{max} = \max_{k \epsilon \{1,2,\cdots,K\}, i \in V_k, j \in V/V_k} d(i,j) \quad (13)$$

B. Simulation results and analysis

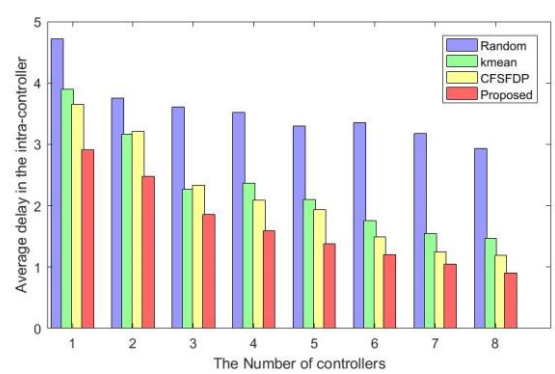

Figure 3. Average delay in the intra-controller

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:01:24 UTC from IEEE Xplore. Restrictions apply.

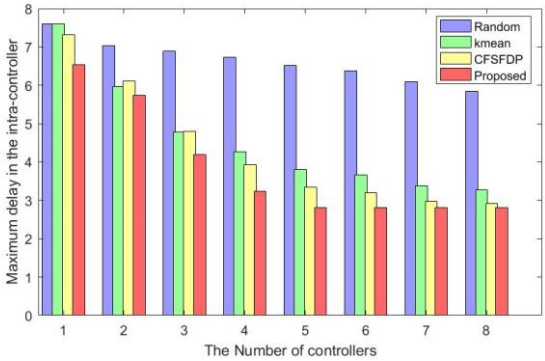

Figure 4. Maximum delay in the intra-controller

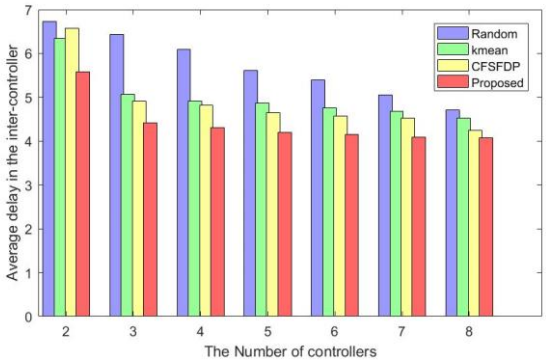

Figure 5. Average delay in the inter-controller

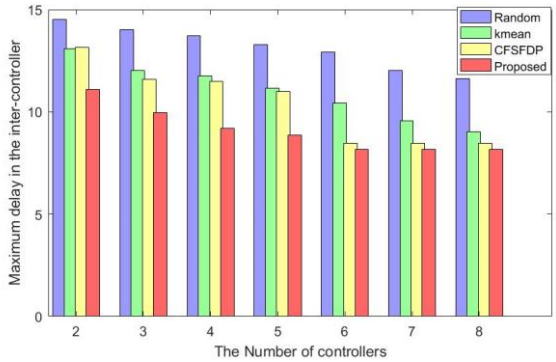

Figure 6. Maximum delay in the inter-controller

Fig. 3 to Fig. 6 show the simulation of our proposed algorithm and the other three algorithms in the average delay in the intra-controller, the maximum delay in the intra-controller, the average delay in the inter-controller and the maximum delay in the inter-controller.

It can be seen from the results that as the number of controllers increases, the propagation delay gradually decreases. This is because as the number of clusters increases, the number of switches in each controller domain gradually decreases, and the controller can better manage the switches in its own domain. The random placement algorithm is the algorithm with the worst performance among the four algorithms compared. This is because all switches are randomly assigned to the controllers and the algorithm does not take into account the distance between switches. The initial cluster centers of K-means are randomly assigned, which causes its performance to be unstable, and the results are different every time. The random placement algorithm also has this problem, so we take the average of 100 runs of the random placement algorithm and the k-means algorithm. The CFSFDP algorithm determines the cluster center by selecting nodes with large values of ρ and δ, thereby determining the location of the controller. So overall it can reduce propagation delay more effectively than k-means algorithm. Our proposed algorithm uses FA to automatically find the d_c value that maximizes the information entropy, thereby increasing the proportion of δ in decision-making. Therefore, the algorithm we proposed can find better cluster centers and determine the location of the controllers, thereby reducing the network propagation delay. We can see from the result data that our proposed algorithm has better results than the other three algorithms from the perspective of reducing propagation delay.

V. Conclusion

In this paper, an SDN controller placement algorithm based on CFSFDP algorithm is proposed. Our proposed algorithm improves the CFSFDP algorithm and applies it to the CPP scenario. We use the information entropy function to automatically determine the important parameter d_c value. We use the distance from the point to the straight line to determine the jumping point, thereby automatically calculate the location and number of SDN controllers in the entire network and the set of switches corresponding to the controllers. The simulation results show that compared with the random placement algorithm, k-means algorithm and CFSFDP algorithm, our proposed algorithm greatly reduces the propagation delay and improves network performance. In the future, we decide to continuously improve our algorithm to improve network reliability.

Acknowledgment

This work is supported by the 111 project (NO.B19007) and Director Funds of Beijing Key Laboratory of Network System Architecture and Convergence (NO.2019BKLNSAC-ZJ-01).

References

[1] F. Yonghong, B. Jun, W. Jianping, C. Ze, W. Ke and L. Min, "A dormant multi-controller model for software defined networking," in China Communications, vol. 11, no. 3, pp. 45-55, March 2014, doi: 10.1109/CC.2014.6825258.

[2] B. Balis, "Increasing Scientific Workflow Programming Productivity with HyperFlow," 2014 9th Workshop on Workflows in Support of Large-Scale Science, New Orleans, LA, 2014, pp. 59-69, doi: 10.1109/WORKS.2014.10.

[3] T. Koponen, M. Casado, N. Gude, J. Stribling, L. Poutievski, M. Zhu, et al., "Onix: a distributed control platform for large-scale production networks," in Usenix Conference on Operating Systems Design and Implementation, 2010, pp. 351-364.

[4] Y. Zhang, N. Beheshti and M. Tatipamula, "On Resilience of Split-Architecture Networks," 2011 IEEE Global Telecommunications Conference - GLOBECOM 2011, Houston, TX, USA, 2011, pp. 1-6, doi: 10.1109/GLOCOM.2011.6134496.

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:01:24 UTC from IEEE Xplore. Restrictions apply.

[5] A. Ksentini, M. Bagaa, T. Taleb and I. Balasingham, "On using bargaining game for Optimal Placement of SDN controllers," 2016 IEEE International Conference on Communications (ICC), Kuala Lumpur, 2016, pp. 1-6, doi: 10.1109/ICC.2016.7511136.

[6] S. Lange et al., "Heuristic Approaches to the Controller Placement Problem in Large Scale SDN Networks," in IEEE Transactions on Network and Service Management, vol. 12, no. 1, pp. 4-17, March 2015, doi: 10.1109/TNSM.2015.2402432.

[7] Y. Fan and T. Ouyang, "Reliability-Aware Controller Placements in Software Defined Networks," 2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems (HPCC/SmartCity/DSS), Zhangjiajie, China, 2019, pp. 2133-2140, doi: 10.1109/HPCC/SmartCity/DSS.2019.00295.

[8] M. F. Bari et al., "Dynamic Controller Provisioning in Software Defined Networks," Proceedings of the 9th International Conference on Network and Service Management (CNSM 2013), Zurich, 2013, pp. 18-25, doi: 10.1109/CNSM.2013.6727805.

[9] J. Liao, H. Sun, J. Wang, Q. Qi, K. Li, and T. Li, ”Density cluster based approach for controller placement problem in large-scale software defined networkings,” Computer Networks, vol. 112, pp. 24-35, 2017.

[10] A. Rodriguez and A. Laio, ”Clustering by fast search and find of density peaks,” Science, vol. 344, no. 6191, pp. 1492-1496, 2014.

[11] H. Xiaolan, W. Muqing and X. Weiyao, "A Controller Placement Algorithm Based on Density Clustering in SDN," 2018 IEEE/CIC International Conference on Communications in China (ICCC), Beijing,China,2018,pp.184-189,doi: 10.1109/ICCChina.2018.8641106.

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:01:24 UTC from IEEE Xplore. Restrictions apply.